RoboWar Programmer's Guide

Web Edition · v1.0 with v0.5 extensions · May 2026

Contents

§1 Overview

RoboWar is a programming game. You write a program that runs autonomously during each battle — you don't control your robot in real time. Programs execute on a **stack machine** (similar to Forth): push values, read sensor registers, write actuator registers, and use control flow to react to the arena.

Each tick (called a *chronon*), every robot's VM runs for a fixed number of cycles determined by its CPU hardware. The arena is 300×300 units. Coordinates are measured from the top-left corner; positive Y goes downward.

Key rule: The stack holds 32-bit integers. All trig functions use degrees and return values scaled by ×1000 (so `SIN 90` returns `1000` meaning 1.000).

§2 Hardware System

Each robot has a **30 hardware-point budget**. Spend points across eight components. Unspent points are wasted — optimise your build for your strategy.

Component	Levels (cost)	Effect
Armor	0(0) · 1(2) · 2(4) · 3(6) · 4(8)	Max HP: 15 / 30 / 50 / 75 / 100
Shield	0(0) · 1(2) · 2(4) · 3(6)	Damage ×1.0 / ×0.75 / ×0.50 / ×0.30; drains 0/2/3/5 energy/tick

Weapon	none(0) · bullet(2) · missile(4) · drone(6) · triple(6)	See weapon table below
Engine	0(0) · 1(2) · 2(4) · 3(6)	Max speed: 4/6/8/12; accel: 1/2/3/4
Energy	0(0) · 1(2) · 2(4) · 3(6)	Max pool: 50/100/150/200; recharge 1/2/3/4/tick
CPU	0(0) · 1(2) · 2(4) · 3(6)	Cycles/tick: 5 / 10 / 20 / 40
Cooling	0(0) · 1(2) · 2(4)	Heat dissipation: 1 / 2 / 4 per tick
Radar	0(0) · 1(2) · 2(4) · 3(6)	Range: 200/350/500/999; cone: 60°/120°/180°/360°

Weapon Table

Type	Cost	Damage	Speed	Heat	Notes
bullet	2	3	15	1	Fast, unlimited ammo
missile	4	8	8	3	Tracks target for 5 ticks, bounces once off walls
drone	6	4	—	5	Stationary, lives 30 ticks; damages enemies that pass through
triple	6	3×3	15	4	3-bullet spread: –12°, 0°, +12° from aim

Tip: Heat caps at 20. If your robot fires until heat ≥ 20 it cannot fire again until it cools. High-rate weapons need Cooling investment.

§3 Language Reference

Programs are plain text. Each line may contain multiple tokens separated by whitespace. Semicolons begin comments that extend to end-of-line.

Stack Machine Basics

All operations use a last-in-first-out stack. Integers are 32-bit signed. The stack holds up to 256 values; overflow drops the oldest entry. An underflowing pop returns 0.

```
5 3 ADD          ; pushes 5, then 3, then pops both and pushes 8
10 DUP MUL      ; pushes 10, duplicates → [10,10], MUL → [100]
```

Arithmetic & Logic

Token(s)	Stack effect	Description
`ADD` `+`	a b → a+b	Integer addition
`SUB` `-`	a b → a−b	Integer subtraction
`MUL` `*`	a b → a×b	32-bit multiply
`DIV` `/`	a b → a÷b	Integer divide, truncates toward zero; b=0 → 0
`MOD`	a b → a mod b	Modulo (always non-negative)
`ABS`	a → |a|	Absolute value
`NEG`	a → −a	Negate
`MAX`	a b → max(a,b)	Maximum
`MIN`	a b → min(a,b)	Minimum
`AND`	a b → 1 or 0	Logical AND (non-zero = true)
`OR`	a b → 1 or 0	Logical OR
`NOT`	a → 0 or 1	Logical NOT
`XOR`	a b → a⊕b	Bitwise XOR
`EQ` `=`	a b → 1 or 0	Equal
`NEQ` `<>`	a b → 1 or 0	Not equal
`LT` `<`	a b → 1 or 0	Less than
`GT` `>`	a b → 1 or 0	Greater than
`LTE` `<=`	a b → 1 or 0	Less than or equal
`GTE` `>=`	a b → 1 or 0	Greater than or equal

Stack Manipulation

Token	Stack effect	Description
`DUP`	a → a a	Duplicate top
`POP` `DROP`	a →	Discard top
`SWAP`	a b → b a	Swap top two
`OVER`	a b → a b a	Copy second-to-top over top
`ROT`	a b c → b c a	Rotate top three: bring third to top

Variables

There are 100 numbered variable slots (1–100). Use `#DEFINE` to create named aliases.

```
#DEFINE speed 1        ; name slot 1 "speed"
42 STORE speed          ; store 42 into slot 1
RECALL speed            ; push slot 1's value (42)
```

§4 Registers

Registers are named I/O ports. **Read-only** sensors push their value onto the stack when you name them. **Write-only** actuators pop a value from the stack and apply it.

Read-Only Sensors

Name	Alias	Range	Description
`ENERGY`		0–maxEnergy	Current energy pool
`ARMOR`		0–maxArmor	Current hit points
`HEAT`		0–20	Weapon heat level (cannot fire when ≥20)
`RANGE`		0–1500	Distance to nearest enemy in SCAN cone; 0 = none detected
`RADAR`		0–359°	Bearing to nearest detected enemy
`SPEEDX`		−maxSpeed–maxSpeed	Horizontal velocity
`SPEEDY`		−maxSpeed–maxSpeed	Vertical velocity
`POSX`	`X`	0–arenaW	X position (distance from left edge)
`POSY`	`Y`	0–arenaH	Y position (distance from top edge)
`COLLISION`		0 or 1	1 if robot collided with wall or another robot last tick
`STUNNED`		0 or 1	1 if robot is currently stunned
`TEAMMATES`	`ROBOTS`	1–n	Total robots currently alive (including self)
`RANDOM`		0–255	Random integer, re-rolled each tick
`TIME`	`CHRONON`	0–tickLimit	Current tick counter
`DAMAGE` v0.5		0–∞	Cumulative damage received since battle start
`DOPPLER` v0.5		integer	Radial velocity of nearest enemy in AIM+LOOK direction; positive = approaching

`TOP` v0.5		0–arenaH	Distance to top wall (= POSY)
`BOT` v0.5		0–arenaH	Distance to bottom wall
`LEFT` v0.5		0–arenaW	Distance to left wall (= POSX)
`RIGHT` v0.5		0–arenaW	Distance to right wall
`ID` v0.5		0–n-1	This robot's 0-based index in the battle

Write-Only Actuators

Name	Value	Description
`AIM`	0–359°	Set gun aim angle in degrees (0=right, 90=down, 180=left, 270=up)
`FIRE`	1 to fire	Fire weapon in current aim direction (ignored if heat ≥ 20 or no weapon)
`SHIELD`	0 or 1	Enable/disable shield (requires shield hardware)
`THRUSTX`	−5 to 5	Horizontal thrust force (clamped)
`THRUSTY`	−5 to 5	Vertical thrust force (clamped)
`BRAKE`	0 or 1	When 1, multiply velocity by 0.80 each tick instead of thrusting
`GUNX`	integer	Set aim by X component of direction vector (used with GUNY)
`GUNY`	integer	Set aim by Y component of direction vector (used with GUNX)
`BEEP`	any non-zero	Queue SIGNAL interrupt on all alive teammates
`LOOK` v0.5	0–359°	Offset DOPPLER scan direction from AIM by this many degrees
`SCAN` v0.5	0–359°	Offset RADAR/RANGE scan direction from AIM by this many degrees

Note: LOOK and SCAN persist across ticks. Set once in your init block and they stay set.

§5 Math & Trig v0.5

All trig functions operate in **degrees** and return values scaled by **×1000**. This avoids floating-point: `SIN 90` returns `1000` (representing 1.0).

Opcode	Stack effect	Description
`SQRT`	a → floor(√a)	Integer square root; negative input → 0
`DIST`	dx dy → floor(hypot)	Euclidean distance from two components
`SIN`	deg → sin×1000	Sine of angle in degrees, ×1000
`COS`	deg → cos×1000	Cosine of angle in degrees, ×1000
`TAN`	deg → tan×1000	Tangent; returns 0 at ±90° (undefined)
`ARCTAN`	y x → deg	atan2(y,x) in degrees, 0–359 range (note: x on top, y below)
`ARCSIN`	val×1000 → deg	Arcsine; input clamped to ±1000
`ARCCOS`	val×1000 → deg	Arccosine; input clamped to ±1000

```
; Compute lead angle for a target 150 units away moving at 3 px/tick sideway
3000 150 DIV   ; sin(angle) × 1000 = 3000/150 = 20
ARCSIN          ; ≈ 1° (lead angle)
```

§6 Control Flow

IF / ELSE / ENDIF

Pops the top value. If it is **non-zero** (true), executes the IF block; otherwise jumps to the ELSE block (or ENDIF if no ELSE).

```
ENERGY 50 >
IF
  1 SHIELD
ELSE
  0 SHIELD
ENDIF
```

LOOP / POOL

Loops unconditionally back to the matching `LOOP`. Programs that reach the end of bytecode also wrap around to the start — a bare `LOOP … POOL` is the most common program structure.

```
LOOP
  RADAR AIM
  1 FIRE
POOL
```

Labels and GOTO

A word ending in `:` defines a label at that bytecode address. `GOTO label` jumps unconditionally. `CALL label` saves the return address; `RETURN` pops it.

```
init:
  SETINT WALL avoidWall
  INTON
  GOTO main

avoidWall:
  5 BRAKE
  RTI

main:
LOOP
  RADAR AIM
  1 FIRE
POOL
```

#DEFINE

Preprocessor macro — expands a name to a value (or another name) before compilation. Useful for naming variable slots and constants.

```
#DEFINE HEALTH_THRESHOLD 30
#DEFINE bearing 1

ENERGY HEALTH_THRESHOLD <
IF
  0 SHIELD   ; conserve energy
ENDIF
```

§7 Interrupt System v0.5

Interrupts let your robot react to events without polling. When a condition fires, the current instruction pointer is saved, interrupts are disabled, and execution jumps to your handler. Call `RTI` at the end to re-enable interrupts and resume normal execution.

Interrupt Opcodes

Opcode	Syntax	Description

`SETINT`	`SETINT type label`	Register *label* as the handler for interrupt *type*. Use `0` as label to disable.
`SETPARAM`	`SETPARAM type value`	Set the threshold parameter for interrupt *type*.
`INTON`		Enable interrupt delivery globally.
`INTOFF`		Disable interrupt delivery (they still queue).
`RTI`		Return from interrupt: re-enable interrupts and restore the saved PC.
`FLUSHINT`		Clear all pending queued interrupts.

Interrupt Types and Default Parameters

Name	Default param	Fires when...
`COLLISION`	—	Robot collided with wall or another robot this tick
`WALL`	30 px	Distance to any wall ≤ param
`DAMAGE`	1 pt	Damage received this tick ≥ param
`SHIELD`	10 energy	Energy ≤ param while shield is active
`TOP`	20 px	Distance to top wall ≤ param
`BOTTOM`	arenaH−20	Distance to bottom wall ≤ param
`LEFT`	20 px	Distance to left wall ≤ param
`RIGHT`	arenaW−20	Distance to right wall ≤ param
`RADAR`	2×arenaW	Nearest detected enemy ≤ param units away
`RANGE`	2×arenaW	Nearest enemy in SCAN direction ≤ param
`ROBOTS`	6	Total robots alive < param (use SETPARAM to tune!)
`SIGNAL`	—	A teammate wrote a non-zero value to BEEP
`CHRONON`	0 (off)	Current tick mod param = 0 (e.g., every 10 ticks)

Warning: The default ROBOTS param is 6. In a 2-robot battle ROBOTS (=2) will immediately be less than 6, so this interrupt fires every tick unless you `SETPARAM ROBOTS 1` or don't register a handler.

Interrupts are **priority-ordered**: lower-numbered types are dispatched first. Only one interrupt fires per tick. Re-queued interrupts from the same condition can fire again next tick if still active.

```
; Register all interrupts in an init block
SETINT WALL     onWall
SETPARAM WALL  60
SETINT DAMAGE  onDamage
SETPARAM DAMAGE 5
SETINT CHRONON every10
SETPARAM CHRONON 10
INTON
GOTO main

onWall:
  1 BRAKE
  RTI

onDamage:
  1 SHIELD
  RTI

every10:
  ; do something every 10 ticks
  RTI

main:
LOOP
  RADAR AIM
  1 FIRE
POOL
```

§8 Example Programs

Simple Duelist

```
; Track enemy, fire, and manage shield based on energy.
LOOP
  RADAR AIM
  1 FIRE
  ENERGY 40 >
  IF
    1 SHIELD
  ELSE
    0 SHIELD
  ENDIF
POOL
```

Doppler Lead-Shot Duelist

```
; Read DOPPLER to estimate lateral velocity and lead the shot.
#DEFINE bearing 1
#DEFINE lead 2

LOOP
  RADAR STORE bearing
  RECALL bearing AIM
  DOPPLER 4 / STORE lead
  RECALL bearing RECALL lead + AIM
  RANGE 0 >
  IF
    1 FIRE
  ENDIF
POOL
```

Wall-Avoiding Interrupt Robot

```
; WALL interrupt fires when within 50 px; steer away from nearest wall.
SETINT WALL wallHandler
SETPARAM WALL 50
INTON

LOOP
  RADAR AIM
  1 FIRE
POOL

wallHandler:
  LEFT 50 <
  IF
    3 THRUSTX
  ELSE
    RIGHT 50 <
    IF
      -3 THRUSTX
    ENDIF
  ENDIF
  TOP 50 <
  IF
    3 THRUSTY
  ELSE
    BOT 50 <
    IF
      -3 THRUSTY
    ENDIF
```

```
  ENDIF
  RTI
```

Reactive Shield Bot

```
; Raise shield when hit by ≥3 damage; lower when energy drops below 15.
SETINT DAMAGE damageHandler
SETPARAM DAMAGE 3
INTON

LOOP
  RADAR AIM
  1 FIRE
  ENERGY 15 <
  IF
    0 SHIELD
  ENDIF
POOL

damageHandler:
  1 SHIELD
  RTI
```

§9 Strategy Tips

Combat

- **Fire only when RANGE > 0** — don't waste heat firing when no enemy is detected.
- **Use DOPPLER to lead shots** — positive DOPPLER means enemy is approaching; subtract a small fraction from the aim angle to lead.
- **SCAN offset** — decouple your radar scan cone from your aim. Useful for scanning a wider arc while aiming at the last known position.
- **Shield timing** — shield is expensive. Use `SETINT DAMAGE` to raise it reactively rather than keeping it on continuously.
- **Missile heat** — missiles generate 3 heat each. With low cooling, you'll jam after 6 shots. Cool down or pair missiles with high-cooling hardware.

Movement

- **BRAKE** is fast deceleration (×0.80/tick). Use it before walls rather than fighting velocity with opposing thrust.

- **Wall sensors** (TOP/BOT/LEFT/RIGHT) give exact distances. A WALL interrupt with SETPARAM 60 gives you time to steer before hitting the border.
- **COLLISION** is reactive (fires after the bounce). Wall sensors are proactive — prefer them for smooth navigation.

Interrupts

- Set up interrupts in an init block before your main LOOP. Use `GOTO main` to skip past handler code.
- Always end handlers with `RTI`. Forgetting RTI leaves interrupts disabled permanently.
- Handlers run with interrupts disabled. Re-entrant interrupts do not fire inside a handler — they queue and fire after RTI.
- `FLUSHINT` clears the queue. Useful in a handler to prevent stale events from firing immediately after RTI.
- **ROBOTS default param is 6** — use `SETPARAM ROBOTS 1` to fire only when you are the last robot alive, or tune to your tournament size.

CPU Budget

- CPU level 1 (10 cycles/tick) is enough for simple programs. Level 2 (20 cycles) handles interrupt-heavy code. Level 3 (40 cycles) enables complex trig and multi-pass logic.
- Count your cycles: each bytecode word is one cycle. SETINT/SETPARAM use 3 words each. Register reads/writes use 2 words. The program wraps at the end.
- If your LOOP body needs more cycles than your CPU has per tick, the VM will resume mid-loop next tick — design accordingly.

RoboWar web edition — vibe coded May 2026 by Michael Morrow using Claude Code.
Original *RoboWar* by Rod McFarland (1989–1994). Additional development by Peter Spear and the RoboWar community.